

The Architectural Test for Instinct/Reasoning Separation: The Discriminating Question That Distinguishes Separation Architectures from Collapsed Architectures in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in *The Instinct/Reasoning Separation Outside the Model* (Li, April 2026), the second paper in the CKS theory series following *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It introduces no new axioms. Its contribution is to formalize a single discriminating question — *Can humans correct AI behavior without modifying model weights or prompt context?* — as the architectural test that operationally distinguishes deployments where the source paper's separation is present from collapsed-architecture deployments where the model is the entire system.

Abstract

Paper 2's foundational claim separates fast-pattern instinct (the LLM, a System-1 analogue) from deliberate reasoning (the CKS substrate inherited from Paper 1, a System-2 analogue) into independently-evolving layers under unified human governance. The source paper's overview contrasts this with conventional LLM-only architectures that "collapse these into one layer where the model is the entire system, which means errors can only be addressed by modifying the model itself." At deployment time the contrast is consequential: the same system can be classified as separation-architecture or collapsed-architecture depending on which corrective pathways are operationally available, and the classification should not depend on rhetorical framing. This note formalizes a single discriminating question — *Can humans correct AI behavior without modifying model weights or prompt context?* — as the architectural test that adjudicates between the two cases. The test is operational rather than theoretical, binary per question (substrate-resident rule authoring suffices, or it does not), and a continuum operationally (deployments may pass for some failure modes and fail for others). The note states the test, distinguishes it from model-centric evaluation, applies it across four practical scenarios, names the Paper 1 commitments it inherits, specifies its scope limits, and provides its operational form.

1. Why the architectural test needs to be formalized as standalone

The instinct/reasoning separation Paper 2 commits to is an architectural property of a deployment, not of the model the deployment uses. The same LLM, integrated under different architectures, can sit inside a separation-conformant system or inside a collapsed system. The deployment's place in this distinction is determined by where corrective authority lives — in substrate-resident rules humans can author, or in components only modifiable through the model layer.

The dominant AI evaluation framing reaches the deployment from a different angle: model-centric metrics — accuracy, calibration, robustness, helpfulness, harmlessness — attached to model artifacts. This vocabulary describes the model well; it describes the architecture poorly. The architectural distinction does not show up in model-centric metrics because they are not asking the architectural question. The remedy is to name that question precisely and ask it operationally — not as a replacement for performance metrics but as a complement: not *how good is the AI* but *how is it corrected when it errs*.

This is the third Phase B2 note decomposing Paper 2's Claim 1, after B2.01 (instinct layer) and B2.02 (reasoning layer). B2.04, B2.05, and B2.06 will formalize routing patterns, high-stakes-decision identification, and verification gates. The architectural test is what makes the prior two standalone definitions verifiable at deployment time.

2. The test, precisely stated

The architectural test for instinct/reasoning separation is one question:

Can humans correct AI behavior without modifying model weights or prompt context?

The question is asked operationally. When an undesired AI behavior is identified in the deployment, the question is whether mechanisms are available to correct that behavior through substrate-resident rule authoring per Paper 1's governance commitments, without modification at the model layer.

Pass condition. The test passes when humans can correct the behavior by adding, modifying, or removing substrate-resident orchestration rules whose authoring is sufficient to change cell processing without further intervention at the model layer. The corrective signal travels through the substrate, exercising Paper 1's rights (inspect, modify, override over substrate content and orchestration rules).

Fail condition. The test fails when correction requires any of: LLM weight modification (fine-tuning, RLHF, RLAIIF, distillation, or other gradient-based update); prompt context modification (prompt engineering, system-prompt edits, few-shot example revision, persona reshaping); modification of retrieval-augmented components held outside substrate authority (RAG embedding updates, corpus replacement, vector-index reconstitution); model swap; or no architectural correction pathway at all.

Binary at the question level; operational rather than theoretical. The deployment either has substrate-resident-rule-authoring as a sufficient correction pathway for the behavior in question, or it does not. The test asks whether substrate-rule authoring is operationally available for actual behavior,

not whether the deployment is described as having a substrate. A deployment that nominally has a substrate but routes all corrections through the model layer fails regardless of architectural description.

For purposes of the test, "the model" is scoped broadly: LLM weights, prompt context, RAG embeddings and corpora outside substrate authority, fine-tuned variants, vendor-supplied agent configurations, and any other component modifiable only through the model-layer pipeline. The discriminating force comes from this broad scoping. A deployment that "corrects" behavior by editing a system prompt is making a model-layer modification, and the test treats it accordingly.

3. What the architectural test is distinctive about

Three properties distinguish the test from model-centric evaluation.

It reframes evaluation from model-centric to architecture-centric. Model-centric evaluation asks how well the model performs across input distributions. The architectural test asks how the deployment corrects errors when they occur. The two questions are independent: a deployment may have a high-performing model and fail the test (because all correction goes through the model layer), and another may have a moderately-performing model and pass the test (because substrate rules suffice). Performance and correctability are orthogonal architectural properties.

It is a deployment-level diagnostic rather than a model-level property test. Two deployments using the same model can produce different test results; one deployment using two different models can pass consistently. The unit of evaluation is the system humans operate, not the model the system embeds.

It is high-leverage — failing it reveals architectural character broadly. A deployment that fails the test has no operationally available substrate-resident corrective pathway; the substrate, if present, is not authoritative for cell behavior in any operationally meaningful sense. This implies multiple Paper 1 commitment violations at once: the substrate is not the source of truth, the LLM is functioning as the source of truth (Paper 1's canonical anti-pattern), and the LLM is holding substrate-relevant state outside the substrate (the mediator role's no-outside-state property violated). One question reveals all three.

4. Practical test scenarios

The test is applied through specific scenarios. Auditors and architects ask it about concrete behaviors rather than abstractly. Four scenarios are typical.

(a) Categorical incorrect outputs. The AI produces incorrect outputs for a specific input category — a particular jurisdiction, product class, or regulatory regime. Test: can substrate-resident rules redirect cell processing for that category — adding compliance gates, mandating verification, selecting different orchestration paths — without retraining or prompt revision?

(b) Compliance gap. The deployment fails a compliance requirement that emerges or intensifies after deployment. Test: can substrate-resident rules add the required checks, structure the required reasoning paths, or pin the required output format, without modifying the model?

(c) Drift over time. The AI's behavior drifts from specifications, or specifications change while the model does not. Test: can substrate-resident rules pin the deployment's behavior to current specifications without weight or prompt modification?

(d) Opaque reasoning. The AI's reasoning over a class of decisions is opaque, and stakeholders need explicit reasoning structures (audit, regulatory disclosure, substantive disagreement). Test: can substrate-resident rules make the reasoning explicit by routing decisions through reasoning-layer processing where steps are substrate-resident and inspectable?

In each scenario, the test resolves to whether the substrate-resident rule-authoring pathway is operationally sufficient. The scenarios are the operational form of the question; the question is the same in every case.

5. Inherited Paper 1 commitments

The test is grounded in Paper 1 commitments Paper 2 inherits without redefense. Naming the inheritance keeps the test from drifting into a stronger claim than the source papers support.

Human-governed authority architecture. Correcting AI behavior is a governance action. The test asks whether the rights Paper 1 commits to — inspect, modify, override over substrate content and orchestration rules at any time — are operationally exercisable for behavior correction. Failing the test indicates the authority is not operationally available even when nominally granted.

Substrate as source of truth. A corrective rule, once written into the substrate, determines subsequent processing. Failing the test indicates that something other than the substrate — most often the LLM and its surrounding pipeline — is authoritative for cell behavior, the canonical anti-pattern Paper 1's source-of-truth note identifies.

Orchestration rule authoring as governance. Paper 1 locates rule authoring as one of two architectural moments at which governance is exercised (the other being direct override). The test asks whether that moment is operationally sufficient for behavior correction. If it is, governance is doing the work the source paper commits it to; if not, the rule-authoring pathway is decorative rather than load-bearing.

AI-as-substrate-mediator. A deployment that fails the test reveals the LLM holding substrate-relevant state outside the substrate, violating the mediator role's no-outside-state property.

The test is the operational specification of these inherited commitments at the deployment level — the question whose answer reveals whether they are operationally present.

6. Operational implications

Three operational implications follow.

Pre-incident architectural diagnosis. The test can be applied before incidents occur. For each anticipated failure mode, ask: *if this happens, what is the correction pathway?* Concrete answers reveal

architectural character without waiting for the failure to materialize.

Failure-mode-specific application reveals partial separation. A deployment may pass for some failure modes (categorical correction available through substrate rules) and fail for others (drift addressable only through retraining; opaque reasoning addressable only through interpretability tooling). Partial separation is an honest characterization of where the deployment instantiates the commitment and where it does not, which a single deployment-wide pass/fail erases.

Procurement vocabulary and upstream coupling. Procurement processes that depend on architectural properties — regulated industries, high-stakes deployments, systems requiring auditable correction — can phrase their architectural requirement as the test, applying it during evaluation rather than only after deployment. The test does not replace operational suites for specific properties; it is the upstream question that gates the rest. Failing it makes downstream tests less informative, because without the separation operationally available, downstream commitments cannot be exercised in the form the source papers specify.

7. Limits

The standalone treatment is not maximalist. Four limits are worth stating explicitly.

Binary at the question level, continuum operationally. The test admits two answers per failure mode, but a deployment is the union of its failure modes. The architectural character is the joint pattern, not a single label. A single deployment-wide pass/fail erases the partial-separation information that per-scenario application surfaces.

Pathway, not quality, of correction. The test asks whether substrate-rule authoring is operationally sufficient as a correction pathway. It does not test whether specific corrections are *correct*, nor whether the substrate or the rules are well-designed. A deployment can pass and apply incorrect corrections through that pathway; the test certifies the architecture, not the rule-author's judgment, and substrate quality is assessed through other operational tests.

Broad scoping of "the model." The test treats LLM weights, prompt context, RAG embeddings outside substrate authority, fine-tuned variants, and vendor-supplied configuration as "the model." Narrower scoping that admitted prompt engineering or RAG-corpus updates as substrate-side corrections would let collapsed deployments pass by definitional sleight — which the source paper's overview specifically forecloses by naming model-modification as the collapsed-architecture's correction mechanism.

Not a replacement for the broader operational test suite. Phase B5 articulates operational tests for the specific commitments Paper 2 makes at Self scope. The architectural test is the most upstream question among that family; the downstream tests verify the specific properties the architecture supports once the separation is operationally present.

8. The architectural test (operational form)

A deployment instantiates the instinct/reasoning separation operationally if and only if the following holds for the failure modes within scope of its operational specification:

- 1 For each in-scope failure mode, an undesired AI behavior of that mode can be corrected by adding, modifying, or removing substrate-resident orchestration rules.
- 2 The correction takes effect through cell processing per the modified substrate, without modifying LLM weights, prompt context, or any model-layer or vendor-supplied component held outside substrate authority.
- 3 The rule-authoring action exercises Paper 1 governance rights — inspect, modify, override over substrate content and orchestration rules — without scheduling, approval, or runtime intermediation that would block the action architecturally.
- 4 The corrected behavior is reproducible across subsequent cell executions: the substrate-resident rule, once written, is authoritative for the behavior it corrects.
- 5 The substrate, not the LLM or its surrounding pipeline, is the source of truth for the cell's behavior in the corrected case.

A deployment that satisfies (1)–(5) for the in-scope failure modes passes the architectural test in those modes. A deployment that fails any of (1)–(5) for any in-scope failure mode fails the test for those modes; the failure indicates the deployment is operating as a collapsed architecture for those modes regardless of description.

9. Conclusion

The instinct/reasoning separation Paper 2 commits to is the foundational architectural move from which the rest of Paper 2 follows: lifecycle primitives, the three-level structure, evolution mechanisms, and multi-shaped governance all operate within the framework the separation establishes. Whether a deployment is operating within that framework, or has collapsed back into the model-as-system pattern Paper 2's overview names as the foil, is a question about actual correction mechanisms, not about how the deployment is described. The architectural test makes the question precise and answerable.

Naming the test as standalone supplies a single high-leverage question for auditors, a precise diagnostic for architects assessing whether the architecture is operationally present rather than nominally claimed, a contracted vocabulary for procurement and regulatory processes, and surfacing of the multi-commitment violation pattern — substrate not source of truth, LLM holding substrate-relevant state, governance not operationally exercisable for behavior correction — that distinguishes a collapsed deployment masquerading as separation from one that holds operationally. This is the third decomposition of Claim 1 in Phase B2; B2.04, B2.05, and B2.06 will formalize the routing patterns, high-stakes-decision identification, and verification gates that complete it. Subsequent work that adopts, extends, or argues against the separation should apply the test in the form formalized here; work using a different test is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Architectural Test for Instinct/Reasoning Separation: The Discriminating Question That Distinguishes Separation Architectures from Collapsed Architectures in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.